Fichas bibliográficas: Fundamentos de sistemas distribuidos y arquitecturas distribuidas

Silva Triviño John Jairo
Sistemas Distribuidos — UTEQ

Noviembre de 2026

Actividad: FORM-FB-01 — Semana 2
Estudiante: Silva Triviño John Jairo

Declaración de uso de inteligencia artificial

Para la elaboración de este trabajo se utilizó Claude (Anthropic) de manera puntual como herramienta de apoyo para aclarar algunas dudas y revisar la redacción de ciertos apartados. La búsqueda de las fuentes, la consulta de los documentos originales y la verificación de sus datos bibliográficos se realizaron mediante la revisión de las publicaciones correspondientes. Los resúmenes fueron elaborados a partir de la lectura y comprensión de las fuentes. También se utilizó la herramienta como apoyo para proponer y mejorar la redacción de las notas personales, las cuales fueron revisadas y adaptadas antes de incorporarlas al documento. La información bibliográfica fue contrastada con las publicaciones originales.

1. Ficha 1

Referencia (IEEE). Y. Abgaz et al., «Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,» *IEEE Transactions on Software Engineering*, vol. 49, n.º 8, págs. 4213-4242, ago. de 2023. DOI: 10.1109/TSE.2023.3287297

DOI / ISBN verificado. DOI: 10.1109/TSE.2023.3287297

Resumen (aprox. 100 palabras). Muchas empresas quieren dividir sus sistemas monolíticos en microservicios, pero los trabajos sobre cómo hacerlo están dispersos y suelen limitarse a casos concretos. Los autores hacen una revisión sistemática: parten de 5022 resultados, aplican cuatro filtros y una ronda de bola de nieve, y analizan 35 estudios publicados entre 2015 y 2021. Con ellos construyen M2MDF, un marco de seis fases que va desde recoger datos del monolito hasta desplegar los servicios. Concluyen que el tema sigue en una etapa temprana: faltan métodos que combinen datos estáticos, dinámicos y evolutivos, herramientas integradas y métricas comunes para comparar propuestas.

Dato concreto. La búsqueda inicial recuperó 5022 estudios; tras cuatro etapas de depuración quedaron 33 y, con la técnica de bola de nieve, 35 *(Sección III-B, pp. 4217–4218).*

Nota personal. Esta investigación me ayudó a entender que pasar de un sistema monolítico a microservicios no consiste solamente en dividir el programa en partes más pequeñas. Es necesario analizar cómo funciona el sistema y decidir qué servicios conviene separar. Me parece importante porque permite comprender mejor las dificultades que puede tener una empresa al modernizar su software. En el TA-IND-01 utilizaré esta fuente para explicar la arquitectura del sistema que analice y los beneficios o problemas que puede presentar una separación por servicios.

2. Ficha 2

Referencia (IEEE). T. Cerný et al., «Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study,» *Journal of Systems and Software*, vol. 206, 2023, art. 111829. DOI: 10.1016/j.jss.2023.111829

DOI / ISBN verificado. DOI: 10.1016/j.jss.2023.111829

Resumen (aprox. 100 palabras). Varios trabajos han estudiado los antipatrones y malos olores de diseño en microservicios, pero usan términos distintos y casi no se solapan, lo que impide tener una visión única. Los autores realizan un estudio terciario: una revisión sistemática que analiza solo estudios secundarios, siete en total, sobre malas prácticas de diseño. A partir de ellos elaboran un catálogo con 58 antipatrones disjuntos en cinco categorías, derivados de 203 identificados al inicio, y reúnen métodos para detectarlos. El resultado sirve de referencia a quienes diseñan microservicios y de base para herramientas que cuiden la calidad mientras el sistema evoluciona.

Dato concreto. El catálogo reúne 58 antipatrones disjuntos en cinco categorías, derivados de 203 antipatrones identificados originalmente en siete estudios secundarios *(Resumen, p. 1)*.

Nota personal. Lo que más me llamó la atención de esta fuente es que utilizar microservicios no garantiza que un sistema esté bien diseñado. También pueden aparecer errores de organización que terminan afectando su funcionamiento y mantenimiento. Esto me permitió comprender que no basta con separar las funciones de una aplicación, sino que también hay que cuidar la forma en que se relacionan los servicios. En el TA-IND-01 me servirá para identificar posibles problemas de diseño y analizar qué tan adecuada es la arquitectura del sistema estudiado.

3. Ficha 3

Referencia (IEEE). G. Zhang et al., «Reaching Consensus in the Byzantine Empire: A Comprehensive Review of BFT Consensus Algorithms,» *ACM Computing Surveys*, vol. 56, n.º 5, ene. de 2024, art. 134. DOI: 10.1145/3636553

DOI / ISBN verificado. DOI: 10.1145/3636553

Resumen (aprox. 100 palabras). Los algoritmos de consenso tolerantes a fallos bizantinos permiten que un sistema distribuido siga funcionando aunque algunos nodos actúen de forma arbitraria, y el auge de blockchain ha multiplicado las propuestas. Entenderlas cuesta porque cada una parte de supuestos distintos y mide la complejidad de otra manera. Los autores revisan de forma sistemática algoritmos influyentes, los clasifican en tres grupos (más eficientes, más robustos y más disponibles) y comparan su complejidad de mensajes y de tiempo. Además, explican cada protocolo paso a paso, con figuras, y señalan sus fortalezas y debilidades frente a los demás.

Dato concreto. En un sistema con $n = 3f + 1$ nodos, el tamaño mínimo de un quórum bizantino es $2f + 1$, frente a $f + 1$ para fallos benignos como los que toleran Paxos y Raft *(Sección 2, "Byzantine quorums", p. 134:5).*

Nota personal. Con esta lectura pude comprender mejor cómo varios equipos o nodos pueden ponerse de acuerdo dentro de un sistema distribuido, incluso cuando algunos presentan fallos o se comportan de manera incorrecta. Antes no tenía muy clara la diferencia entre los fallos normales y los bizantinos. Considero que este tema es importante porque ayuda a entender cómo se busca mantener la confianza y la continuidad de un sistema. Lo utilizaré en el TA-IND-01 como parte del análisis de tolerancia a fallos y confiabilidad, sin asumir que el sistema estudiado utiliza necesariamente este protocolo.

4. Ficha 4

Referencia (IEEE). A. Singh et al., «A survey and taxonomy of consensus protocols for blockchains,» *Journal of Systems Architecture*, vol. 127, 2022, art. 102503. DOI: 10.1016/j.sysarc.2022.102503

DOI / ISBN verificado. DOI: 10.1016/j.sysarc.2022.102503

Resumen (aprox. 100 palabras). Los protocolos de consenso son piezas clave de las cadenas de bloques, pero muchas versiones actuales consumen demasiados recursos, son complejas de configurar o tienen brechas de seguridad, y no encajan en entornos limitados como el Internet de las cosas. Los autores presentan una revisión con una taxonomía de estos protocolos. Parten de sus orígenes (Proof-of-X, tolerancia a fallos bizantinos, Paxos y Raft), incluyen variantes basadas en grafos acíclicos dirigidos y analizan ventajas, desventajas y aplicabilidad en IoT. Cierran con tendencias de investigación y problemas abiertos, y afirman ofrecer un resumen más completo que otras revisiones del área.

Dato concreto. La revisión parte de los protocolos Proof-of-X, tolerancia a fallos bizantinos, Paxos y Raft, y añade variantes basadas en grafos acíclicos dirigidos (DAG), evaluando su aplicabilidad en IoT *(Resumen, p. 1)*.

Nota personal. Esta fuente me permitió conocer que no todos los protocolos de consenso funcionan de la misma manera y que su elección depende de las características del entorno donde se utilizan. Me pareció interesante la relación con el Internet de las cosas, porque los dispositivos pueden tener recursos limitados y no siempre pueden trabajar con protocolos que consumen demasiado. En el TA-IND-01 aprovecharé esta información para comprender mejor la importancia de elegir una arquitectura y mecanismos de comunicación adecuados según las necesidades del sistema.

5. Ficha 5

Referencia (IEEE). M. Kleppmann y C. Riccomini, *Designing Data-Intensive Applications, The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 2.ª ed. Sebastopol, CA: O'Reilly Media, 2026, ISBN: 978-1-0981-1906-5

DOI / ISBN verificado. ISBN: 978-1-0981-1906-5 (edición impresa)

Resumen (aprox. 100 palabras). Hoy muchas aplicaciones dependen más de los datos que de la potencia de cálculo, y sus retos son la escalabilidad, la consistencia, la fiabilidad, la eficiencia y el mantenimiento. El libro aborda ese problema comparando ventajas e inconvenientes de distintas tecnologías para almacenar y procesar datos, e incorpora en esta segunda edición tecnologías y tendencias recientes. Su método es explicar los principios y la investigación en sistemas distribuidos sobre los que se construyen las bases de datos modernas, apoyándose en la arquitectura de servicios en línea reales. Así, ingenieros y arquitectos pueden entender los fundamentos y elegir con criterio entre alternativas.

Dato concreto. Una aplicación es intensiva en datos cuando la gestión de los datos es uno de sus principales retos; en las intensivas en cómputo, en cambio, el reto es paralelizar un cálculo muy grande *(Capítulo 1, "Trade-Offs in Data Systems Architecture").*

Nota personal. Este libro me ayudó a relacionar los sistemas distribuidos con algo que está presente en muchas aplicaciones: el manejo de grandes cantidades de datos. Entendí que no solamente importa almacenar la información, también hay que procurar que esté disponible, que sea confiable y que pueda consultarse sin afectar el funcionamiento del sistema. En el TA-IND-01 utilizaré estos conocimientos para analizar cómo se administra la información y qué importancia tienen la escalabilidad, la consistencia y la disponibilidad de los datos.

Referencias

[1] Y. Abgaz et al., «Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,» *IEEE Transactions on Software Engineering*, vol. 49, n.º 8, págs. 4213-4242, ago. de 2023. DOI: 10.1109/TSE.2023.3287297

[2] T. Cerný, A. S. Abdelfattah, A. A. Maruf, A. Janes y D. Taibi, «Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study,» *Journal of Systems and Software*, vol. 206, 2023, art. 111829. DOI: 10.1016/j.jss.2023.111829

[3] G. Zhang et al., «Reaching Consensus in the Byzantine Empire: A Comprehensive Review of BFT Consensus Algorithms,» *ACM Computing Surveys*, vol. 56, n.º 5, ene. de 2024, art. 134. DOI: 10.1145/3636553

[4] A. Singh, G. Kumar, R. Saha, M. Conti, M. Alazab y R. Thomas, «A survey and taxonomy of consensus protocols for blockchains,» *Journal of Systems Architecture*, vol. 127, 2022, art. 102503. DOI: 10.1016/j.sysarc.2022.102503

[5] M. Kleppmann y C. Riccomini, *Designing Data-Intensive Applications, The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 2.ª ed. Sebastopol, CA: O'Reilly Media, 2026, ISBN: 978-1-0981-1906-5.